Temasek POLYTECHNIC

School of Informatics & IT

Application Development Project (40%)

Database Application Development (30%)

(System Code Implementation)

AY 24/25 Oct Semester

Project 2 Report

Contents

Temasek POLYTECHNIC

SCHOOL OF INFORMATICS & IT

Diploma in [Common ICT]

Project 2 Report

Student Name (Matric Number)	: Mohamed Tawfeeq (2400212C)
Tutorial Group	: P15
Tutor	: Sathish SRITHARAN
Submission Date	: 24/1/2025

Declaration of Originality

I am the originator of this work and I have appropriately acknowledged all other original sources used as my references for this work.

I understand that Plagiarism is the act of taking and using the whole or any part of another person's work, including work generated by AI, and presenting it as my own.

I understand that Plagiarism is an academic offence and if I am found to have committed or abetted the offence of plagiarism in relation to this submitted work, disciplinary action will be enforced.

<u>Declaration on the use of Generative AI tools for assignments</u>

Describe how you have used Generative AI tools such as ChatGPT or Dall.E- in your assignment.

Share the link to the conversations you had with the AI tool (i.e., the prompts you used and the responses you get from the AI tool).

Please refer to this PDF on "How to share the conversations made with ChatGPT?"

How to share the conversations made v

<u>How do you indicate the reference?</u>

The content generated by AI tools are not retrievable except by the user who generated them, so they are considered non-recoverable sources. Although non-recoverable data or quotations in APA Style papers are usually cited as personal communications, with ChatGPT-generated text there is no person communicating. Quoting text from ChatGPT chat is therefore more like sharing the output of an algorithm, with a reference list entry and the corresponding in-text citation.

According to the official APA Style site, ChatGPT references should be cited as:

E.g. OpenAI. (2024). *ChatGPT* (Feb 13 version) [Large language model]. https://chat.openai.com/chat

Note: The information in parentheses refers to the update or revision date of the model used. Refer to the release notes in the ChatGPT application.

Important Note:

- Do not copy answers produced by the AI tool in totality as it is considered as plagiarism.

- Do not rely on any information produced by the AI tool blindly. You should always verify the answer with other sources. Do not assume that these answers provided by the AI tool are correct.

- To achieve quality outputs from the AI tool, you should provide good prompt that is clear and specific. Be precise and provide context. Avoid asking open-ended questions.

ADEV Part 2: System Code Implementation (40%)

Frontend Implementation (25%)

List down all the URL to the different pages of the website.
Provide the sample input values if any.

URL	Purpose	Inputs
http://127.0.0.1:8080/vending_machine.html	This page displays every single vending machine in Temasek Polytechnic with its location details.	
http://127.0.0.1:8080/vending_item.html?id=?	This page will display the items inside each vending machine uniquely according to its vending machine id which can be found in the url. It is a Dynamic ID E.g id = 1 will show items in vending machine id 1	
http://127.0.0.1:8080/addvendingitems.html?id=?	This page brings up a form which allows the user to input and add items inside the specific vending machine based off the vending machine id in the URL. It is a	Item Name: 100 Plus Item Cost:1.20 Item Quantity:10 Item Image URL : 100plus.jpeg Item Availability: Available

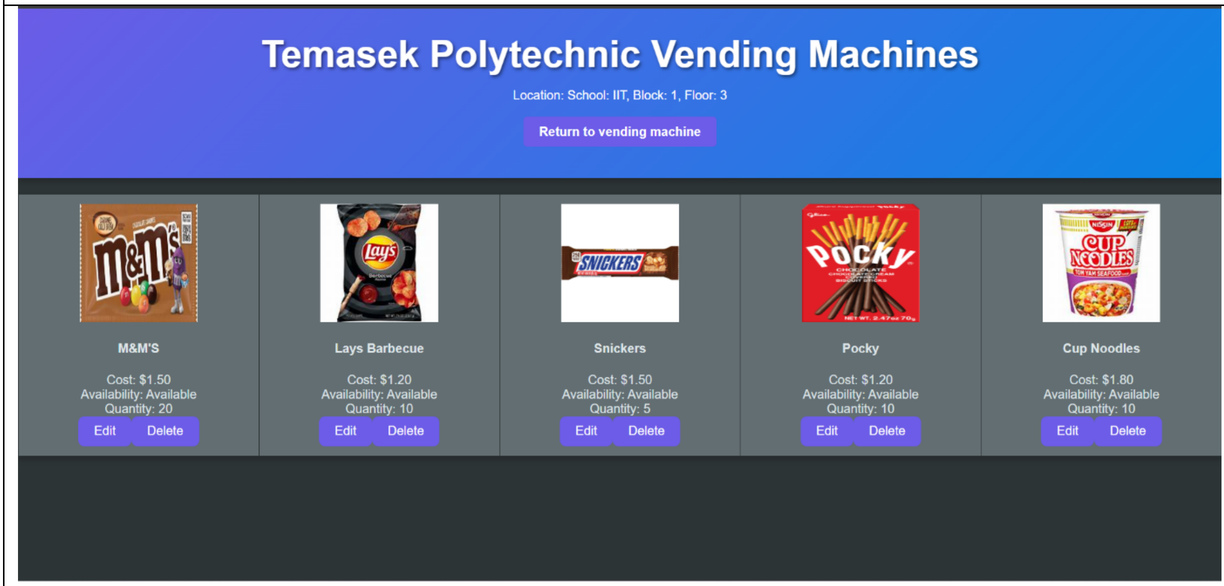
	Dynamic ID E.g. id = 1 will ensure when you add an item it will add inside the correct vending machine.	Vending Machine ID:1(read-only) Vending Item ID:100
http://127.0.0.1:8080/update_items.html?id=?	This page will bring up a form with the item details by the id in the URL which will allow the user to update the item by adjusting the details they want to change and press the update button. It is a Dynamic ID E.g. id=1 will update the item information by item_id in the item table.	Item ID: 1 (read-only) Item Name: 100 Plus Item Cost:1.50 Item Quantity:0 Item Image URL : 100plus.jpeg Item Availability: Unavailable
http://127.0.0.1:8080/low_stock.html	This is one of my additional features. This page will show all the items that are low on stock so that the admin is able to view which items need to be restocked.	

User Interface (10%)

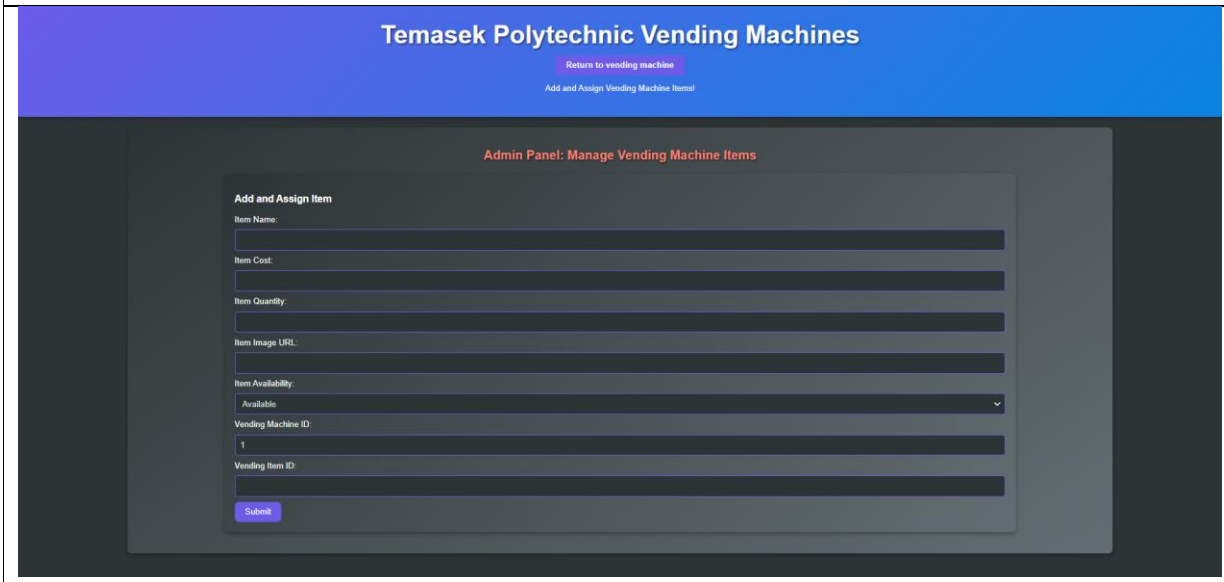
Provide screenshot(s) of the user interface for each of the core features.
Add more tables if required.

Feature: To view all the vending machines in Temasek Polytechnic

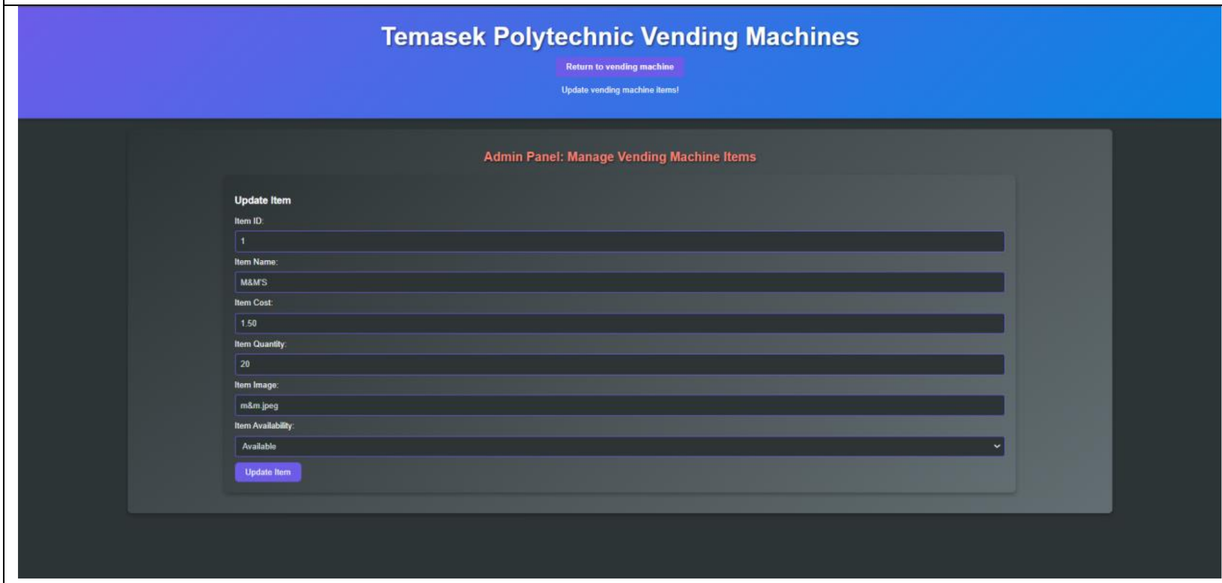

Feature: To view all the items in the specific vending machine. It also shows the location details of the vending machine in the header. This can be accessed by pressing the "View Items" in the vending machine page. The user can return to the vending page when they click on "Return to vending machine".



Feature: To add an item to a specific vending machine. This can be accessed by pressing the "Add Item" in the vending machine page.



Feature: To edit a specific item in a specific vending machine. This can be accessed by pressing the "Edit" button below the items that are inside the vending machine.



Feature: To delete a specific item from a specific vending machine

Additional Features (5%)

Provide screenshot(s) of the additional features.
Explain how each additional feature enhances the website. Write a report on <u>no more than **150 words**</u> PER mechanism:

[Add more tables if required.]

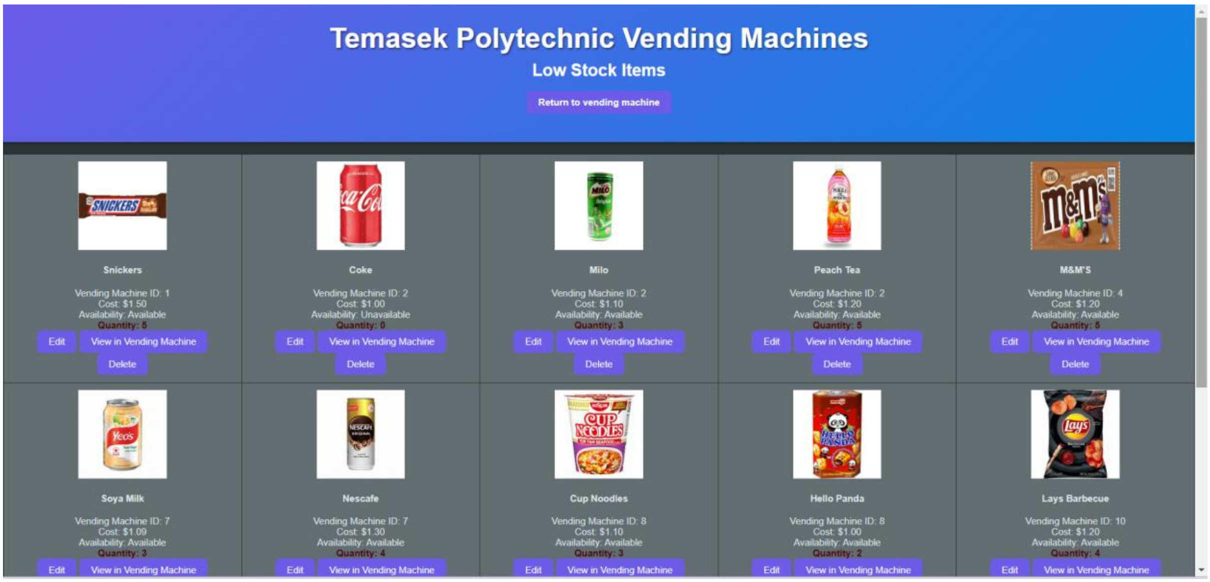
Additional Feature: Filter Vending Machine by Schools

Screenshots:

Explanation: In the main page of the vending machine, it will display all the vending machines in Temasek Polytechnic. This filter function enables the user to only see the vending machines by which school they are located at. This will show a more organised way of displaying the vending machines and allows the user to better visualise the vending machines by school rather than searching for it in the main page with many different vending machines. It puts it in a more organised format which makes it easier for the user to identify the vending machines and locate them quickly according to their needs. This can be useful to the user as it will allow them to save time when locating a vending machine and to avoid any confusion.

Additional Feature: Show Low Stock Items

Screenshots:



Explanation: The Low Stock feature allows the user to view items that are low on stock, which means item quantity less than or equal to 5. This way, the user can keep track of the items that have sold out or about to be sold out so that they can restock them accordingly, rather than individually checking each vending machine. The user can press the "Low Stock Items" in the main page to bring them to the page that shows the items. It displays the vending machine id that each item is inside, and they are also able to view the items in that vending machine when they press the "View in Vending Machine" button. Once they have updated the item quantity by using the edit feature under each item, it will not show in this page anymore as they will have restocked the items.

DBAV Part 2: System Code Implementation (30%)

API Implementation (16%)

Write out the related SQL statement, route pattern, HTTP method and payload (if any) for every functionality that your web application has. You will demonstrate to your Lecturer if the functionalities work using Postman based on the routes you created in the route file, and the functions that you wrote in the controller and DB files.

Functionalities of Web Application	Related SQL code (Full and complete SQL Statements, not partial)	URL Route Pattern	Payload	HTTP Method	Result From Postman (Pass/Fail) (To be filled by Lecturer)
Get all the items	`SELECT * FROM VENDING_MACHINE.ITEM`	/item	N.A	GET	
Get all items by item ID	`SELECT * FROM VENDING_MACHINE.ITEM WHERE item_id = ?;`	/item/:id	N.A.	GET	

Get all the vending machines	`SELECT`
	`vending_machine.vending_machine_id,`
	`vending_machine.vendor_name,`
	`status.status_name,`
	`location.school,`
	`location.block,`
	`location.floor,`
	`GROUP_CONCAT(payment_method.payment_name SEPARATOR ', ') AS payment_methods`
	`FROM`
	`vending_machine`
	`JOIN`
	`location ON vending_machine.location_id = location.location_id`
	`JOIN`
	`vending_payment ON vending_machine.vending_machine_id = vending_payment.vending_id`
	`JOIN`
		/combine	N.A	GET	

	payment_method ON vending_payment.payment_id = payment_method.payment_id JOIN status ON vending_machine.status_id = status.status_id GROUP BY vending_machine.vending_machine_id, vending_machine.vendor_name, status.status_name, location.school, location.block, location.floor;				
Get the vending machine by its ID	SELECT vending_machine.vending_machine_id, vending_machine.vendor_name, status.status_name, location.school, location.block, location.floor, payment_method.payment_name FROM vending_machine JOIN location ON vending_machine.location_id = location.location_id JOIN vending_payment ON vending_machine.vending_machine_id = vending_payment.vending_id JOIN payment_method ON vending_payment.payment_id = payment_method.payment_id JOIN status ON vending_machine.status_id = status.status_id	/combine/:id	N.A	GET	

	WHERE vending_machine.vending_machine_id = ?;				
Get all the items from all vending machines	SELECT vending_machine.vending_machine_id, vending_machine.vendor_name, item.item_id, item.item_name, item.item_cost, item.item_quantity, item.availability, item.item_image, vending_item.vending_item_id FROM vending_machine JOIN vending_item ON vending_machine.vending_machine_id = vending_item.vending_machine_id JOIN item ON vending_item.item_id = item.item_id;	/vending	N.A	GET	
Get all the items inside the vending machine by its vending machine ID	SELECT vending_machine.vending_machine_id, vending_machine.vendor_name, item.item_id, item.item_name, item.item_cost, item.item_quantity, item.availability, item.item_image,	/vending/:id	N.A	GET	

	vending_item.vending_item_id FROM vending_machine JOIN vending_item ON vending_machine.vending_machine_id = vending_item.vending_machine_id JOIN item ON vending_item.item_id = item.item_id WHERE vending_machine.vending_machine_id = ?;				
Add an item inside a vending machine	INSERT INTO VENDING_MACHINE.ITEM (item_name, item_cost, item_image, availability, item_quantity) VALUES (?, ?, ?, ?, ?); INSERT INTO VENDING_MACHINE.VENDING_ITEM (vending_item_id, vending_machine_id, item_id) VALUES (?, ?, ?);	/add_vending_item	{ "item_name": "Snickers", "item_cost": "1.50", "item_quantity": 5, "availability": 1, "item_image": "snickers.jpeg", "vending_machine_id": 1, "vending_item_id": 99 }	POST	
Update an item by its Item ID	UPDATE VENDING_MACHINE.ITEM SET item_name = ?, item_cost = ?, item_image = ?, availability = ?, item_quantity = ? WHERE item_id = ?;	/item/:id	{ "item_name": "M&M'S", "item_cost": "1.50", "item_image": "m&m.jpeg", "availability": 1, "item_quantity": 20 }	PUT	

Delete an item	DELETE FROM VENDING_MACHINE.ITEM WHERE item_id = ?;	/item/:id	N.A	DELETE	

Additional Features (4%)

Write out the related SQL statement, route pattern, HTTP method and payload (if any) for the additional features in ADEV Part 2:

Functionalities of Web Application	Related SQL code (Full and complete SQL Statements, not partial)	URL Route Pattern	Payload	HTTP Method	Result From Postman (Pass/Fail) (To be filled by Lecturer)
Filter Vending Machine by Schools	SELECT vending_machine.vending_machine_id, vending_machine.vendor_name, status.status_name, -- Correctly fetches the status name location.school, location.block, location.floor, GROUP_CONCAT(payment_method.payment_name SEPARATOR ', ') AS payment_methods FROM vending_machine JOIN location ON vending_machine.location_id = location.location_id JOIN vending_payment ON vending_machine.vending_machine_id = vending_payment.vending_id JOIN payment_method ON vending_payment.payment_id = payment_method.payment_id JOIN status ON vending_machine.status_id = status.status_id GROUP BY vending_machine.vending_machine_id,	/combine	N.A	GET	

	vending_machine.vendor_name, status.status_name, location.school, location.block, location.floor;				
Get the low stock items and the vending machine it is in.	SELECT vending_machine.vending_machine_id, vending_machine.vendor_name, item.item_id, item.item_name, item.item_cost, item.item_quantity, item.availability, item.item_image, vending_item.vending_item_id FROM vending_machine JOIN vending_item ON vending_machine.vending_machine_id = vending_item.vending_machine_id JOIN item ON vending_item.item_id = item.item_id WHERE item.item_quantity <= 5; `;	/vending_machine/lowstock	N.A	GET	

Error Handling (10%)

Explain how you implemented the 2 (TWO) error-handling mechanisms on your server-side system, and why they are important. Write a report on <u>no more than **200 words**</u> PER mechanism:

Error handling 1: HTTP Status Code
How you implemented: In my server-side system, I implemented HTTP status to code the show the response of different API requests. If an item has been added successfully or to get information from the data base, the API call will work and show a "200 OK" status code. If there are missing fields in the post function, it will return a "400 Bad Request". If the data is not found it will return a "404 Not Found" If there is an error in database connection or query, it will return a "500 Internal Server Error". It is implemented in "if error, return status ('the HTTP status of the error.')".
Importance: When an item has been added successfully, in the front-end portion it will redirect the user to the page and displays the item. If the item has not been added it will pop up an error message and it will stay in the form screen to avoid confusion for the user. This is possible due to the HTTP status code server-side implementation which handles all the backend error handling which can be displayed on the frontend as this is the portion that connects to the database.

Error handling 2: Try-catch error
How you implemented: I added try-catch blocks in multiple routes to catch runtime errors. Let's say I want to update an item, the try block will have the SQL statement that updates into the database by its id, name etc. If there were to be any error in that try block, the catch block will handle the error and show an error message," Failed to update item" This will ensure that in the event of an error the server will not crash.
Importance: It will prevent the server from crashing and ensures the website is still up and running. Any issues such as network errors or missing data are managed properly and without this the server can crash and the website will not work anymore. This is important as the user still needs to use the website without it crashing as it can be an unpleasant user experience. In the event of an error, the system can recover and make it pleasant for the user by allowing the frontend to display error messages when the error occurs instead of entirely crashing the server.